🤖 AI Usage Note & Development Log

Project Name:	Celare - PII Detector & Masker
Team Name:	Kinetic Prism
Date:	June 9, 2026
Primary AI Assistant:	Trae AI Assistant (Powered by LLMs)

1. AI Development Strategy

Our team utilized an AI Pair-Programming (Driver/Navigator) framework within a shared live access folder environment. One team member acted as the "Driver," engineering specific, context-rich prompts to guide code generation, while the remaining members acted as "Navigators," performing immediate code quality reviews, checking syntax execution outputs, and ensuring local architectural compliance. Every script and design file was generated, validated, and consolidated directly inside the shared team workspace directory.

2. System Prompt Strategy

We utilized iterative, role-based prompting to build the platform layer by layer:

> **DevOps Engineer Persona:** Used to establish the workspace folder architecture, set up operational environments, and handle remote connection configurations.
>
> **Python Data Engineer Persona:** Invoked to develop the backend data pipelines, construct data manipulation scripts, and engineer strict string-matching functions.
>
> **AI Agent Architect & QA Engineer Persona:** Employed to implement the contextual logic loops with local inference engines and write comprehensive integration test files.
>
> **UI Engineer & Technical Writer Persona:** Leveraged to construct the interactive web console layout, map analytics graphs, and write operational documentation.

3. Structural Successes

Modular Architecture: Complete separation of concerns across functional files (data_ingestion.py, regex_engine.py, llm_engine.py, and app.py).

Dynamic Structural Masking: Successfully built targeted replacement structures that preserve structural properties (such as leaving trailing characters visible) using regular expression capture groups.

Data Validation Pipeline: Integrated automated internal checking scripts (run_tests.py and test_pii.py) to confirm zero-crash operations during high-speed data ingestion.

Stateful Presentation Design: Implemented persistent dashboard metrics, real-time file processing status bars, and dynamic multi-format export panels using custom workspace tracking parameters.

4. Edge-Case Errors & AI Hallucinations Corrected

Pandas Float Artifact Bug: The AI's initial pattern-matching checks failed on telephone records because the data-loading scripts automatically classified long digit strings as floats, adding an trailing fraction value to the numbers. **Fix:** Overrode the engine with an upstream string-cast cleaner: `df.astype(str).replace(r'\.0$', '', regex=True)`.

Interface Rendering Failures: The AI initially hallucinated complex layout formatting configurations using unstable absolute CSS overrides, which crashed the underlying user interface window framework on launch. **Fix:** Reverted to native workspace container columns and structured layout blocks.

Separated Data Skipping: Early automated regular expression patterns relied on strict word boundaries that failed when tracking identity rows containing custom spacing or hyphens. **Fix:** Engineered an explicit negative lookahead and lookbehind schema combined with non-capturing groups to enforce data extraction regardless of text formatting combinations.

5. Core Prompts Log

Below is the condensed summary of the high-leverage prompts engineered to build our application suite directly inside our shared live access folder.

🛠️ Workspace Inception & Initialization

```
"Act as a DevOps architect. I am hosting a local Python workspace that
developers will connect to remotely via Live Share. We are building a PII
Detector and Masker strictly using AI generation. Generate a highly strict
exclusion template file, three distinct isolated Python script foundations
(data_ingestion.py, regex_engine.py, llm_engine.py) with complete docstrings,
a cooperation rule file, and a base requirements configuration listing
pandas, pytest, streamlit, and ollama."
```

🧠 Core Data Processing & Pattern Masking

```
"Act as a Python Data Engineer. Implement the full code for data processing
and structural masking with zero placeholders in our shared folder. Build
functions using pandas to load and export data files safely while handling
encoding variations. Write regular expression patterns to match and mask
specific structured entity strings (Emails, Phone Numbers, PAN cards, and
Aadhaar variations) and expose a master pipeline function that tracks metric
counters across all evaluations."
```

🧪 Contextual Inference & Validation Integration

```
"Act as an AI Agent Architect and QA Engineer. Implement the complete
contextual matching logic inside our shared local environment. Construct an
autonomous loop utilizing a local Ollama model instance to process
unstructured text data. If extraction confidence indicators report less than
85%, trigger up to 3 iterative verification reprompts with surrounding
contextual tokens. Generate an automated verification test script covering
all validation parameters."
```

🖥️ Frontend Console & Metric Visualization

```
"Act as an expert Frontend Engineer and UI/UX Designer. Re-architect the
application layout using custom workspace parameters to display a clean,
high-conversion interface. Position the primary document upload zone
prominently at the center, integrating live evaluation processing charts,
interactive highlighting hooks tied to summary data counters, a persistent
run-history tracking tab, and dedicated multi-format file download buttons."
```

🎯 Parser Hardening & Structural Redaction Fixes

```
"Act as a Senior Python Debugger. Update the text extraction engine to
eliminate row-skipping errors on unheaded document files. Cast datasets
explicitly to string types and apply an automated regex replacement matrix
using a non-capturing group format to strip layout delimiters while
preserving start and end indicators. Ensure alignment across data
presentation matrices, metrics panels, and multi-extension exporter
endpoints."
```

6. Human-in-the-Loop & Model Limitations

All AI-generated backend scripts, layout containers, and configuration requirements were manually cross-examined, compiled, and validated by the team. Core operational logic boundaries have been locked down strictly to regional enterprise data definitions to maintain hyper-focused execution paths and guarantee minimal false-positive rates during local high-speed data auditing sweeps.